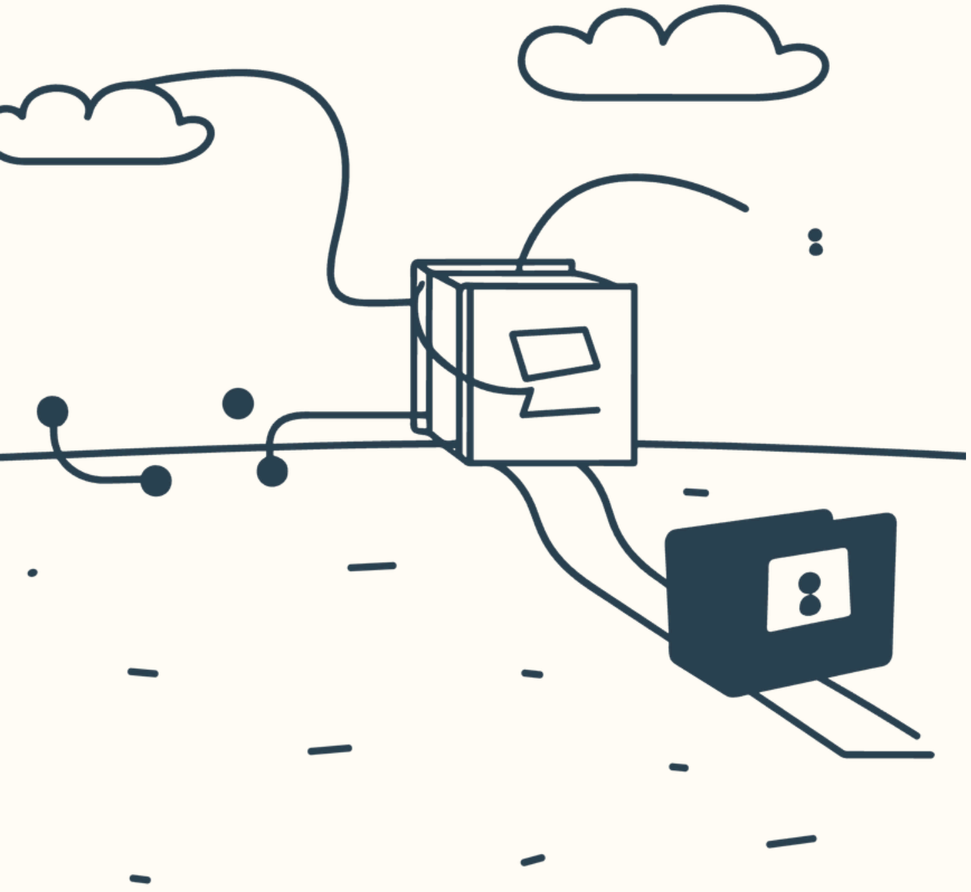


End2End

An End-to-End Encrypted Messaging Application

Presented By,

Dhanjyoti Das (US-231-023-0080)

Darshan Bhattacharjee (US-231-023-0080)

Institution: Lalit Chandra Bharali College

Guide: Mrs. Manisha deka

THE PROBLEM

1. Most messaging platforms collect user metadata such as phone numbers, contacts, IP logs, and activity information.
2. User privacy depends on trusting the service provider.
3. Even with E2EE, metadata can reveal communication patterns.

OUR SOLUTION

1. Session-inspired privacy-first authentication.
2. No phone number , emailid or personal information required.
3. End-to-end encrypted communication with client-owned keys.
4. No metadata collection and ciphertext-only message storage.

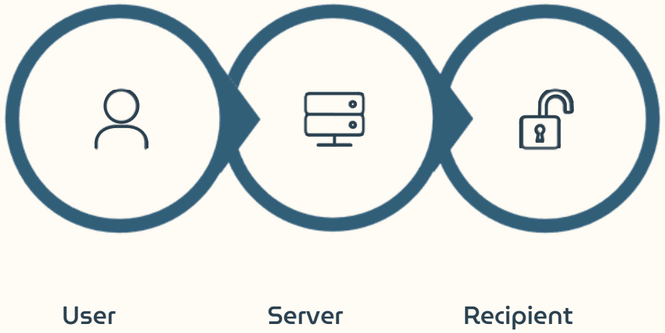
Introduction

End2End is a privacy-focused web messaging application designed to provide secure and anonymous communication.

Privacy-first authentication with no email or phone number required; account recovery is secured through a 12-word mnemonic phrase.

It employs end-to-end encryption to ensure that only the intended sender and recipient can access message contents.

Private encryption keys remain on the user's device, while the server stores only encrypted data and facilitates message delivery.



Project Objectives

1 **End-to-End Confidentiality**

Provide end-to-end confidentiality for all messages — only the intended recipient can decrypt.

2 **Real-Time Delivery**

Real-time message delivery with WebSockets ensuring instant synchronisation.

3 **Secure Key Management**

Secure key management in the client using IndexedDB for local persistence.

4 **Usable Web UI**

Usable web UI built with Next.js and Tailwind CSS for a polished experience.

Technology Stack

Frontend

Next.js (App Router), Tailwind CSS, React + Zustand for state management.

Backend

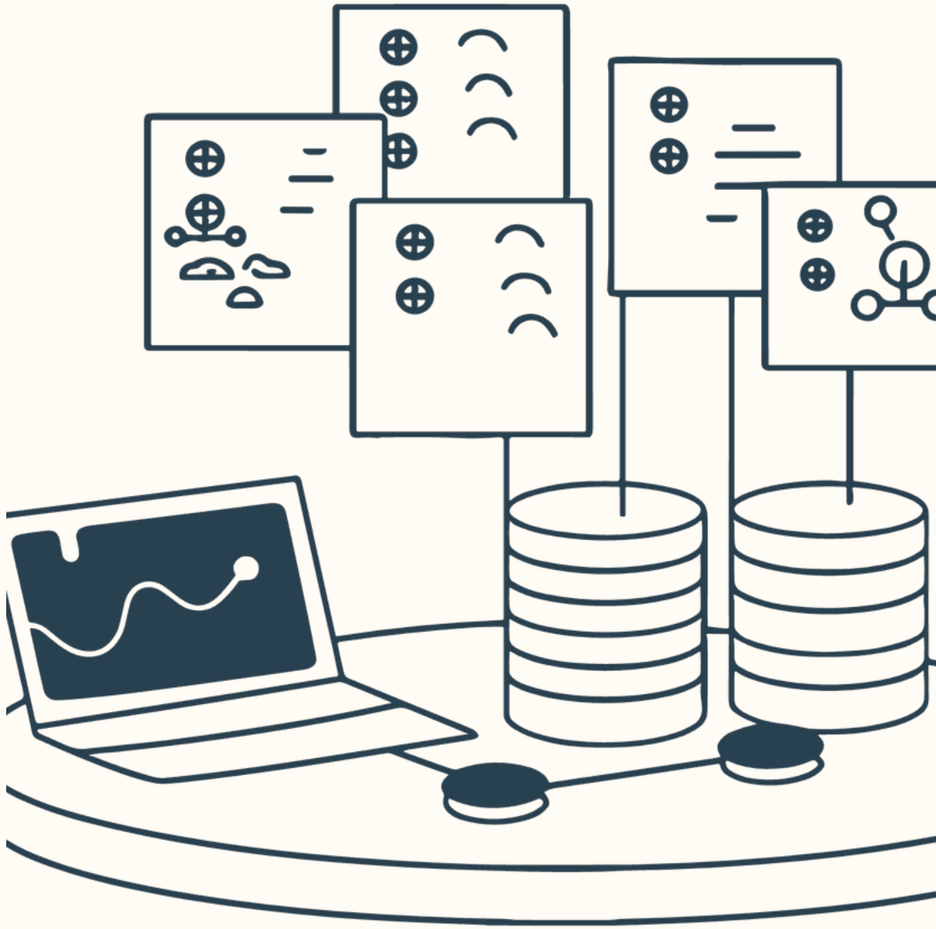
NestJS, Socket.IO gateway, Prisma ORM with PostgreSQL.

Cryptography

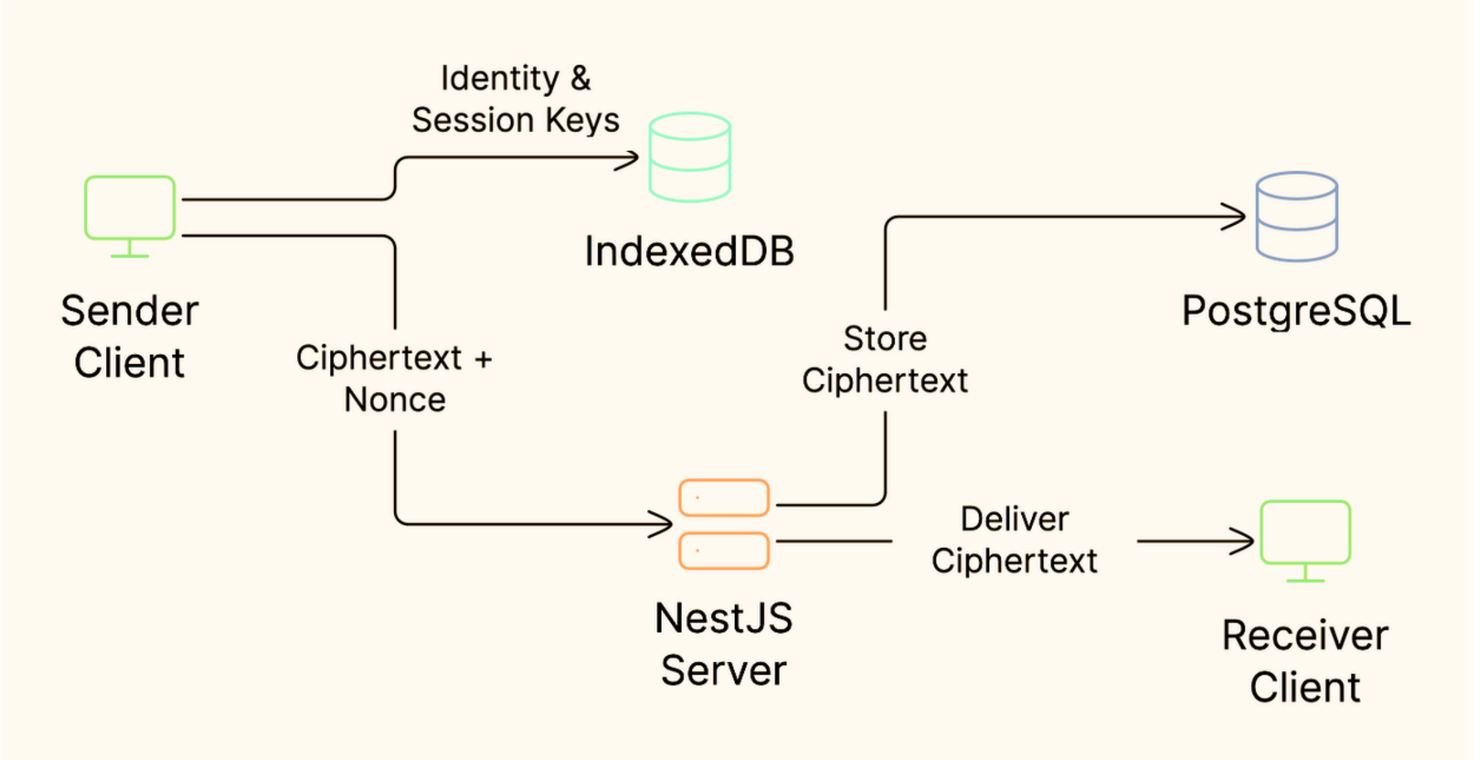
libsodium-wrappers (XChaCha20-Poly1305), libsignal dependency present for session protocol.

Storage

PostgreSQL (server data), IndexedDB (client sessions and identity keys).



System Architecture



- **Client (Next.js)** manages keys, encryption, UI, and socket connection
- **Server (NestJS)** handles auth, message persistence, and Socket gateway
- **IndexedDB** stores identity and session keys locally on the client
- **Postgresql DB** stores only ciphertext + nonce in the Message table

Frontend Implementation

Application Architecture :

- Built using Next.js App Router architecture
- Modular and reusable component-based design
- Clear separation of UI, logic, and data layers

Security & Encryption :

- End-to-end message encryption and decryption
- Client-side key generation and management
- Secure handling of cryptographic operations using Libsodium

State Management :

- Zustand stores for centralized authentication and chat state management
- IndexedDB for persistent local storage of cryptographic keys and user data
- Custom React hooks for reusable business logic and state access

Real-Time Communication :

- WebSocket-based real-time messaging with Socket.IO
- Authenticated and secure client-server connections
- Instant message delivery and live updates

Backend Implementation

Modular Architecture :

- Built using NestJS modular architecture
- Clear separation of responsibilities
- Scalable and maintainable backend design

Authentication Module :

- User authentication and Authorization
- JWT-based access control
- Secure cookie and session management

Key Management Module:

- Storage and retrieval of user public keys
- Identity verification support
- Secure key lifecycle management

WebSocket Gateway

- Real-time client-server communication
- Connection authentication and authorization
- Room management and event broadcasting

Conversation Module :

- Message and conversation management
- Transactional data persistence
- Reliable storage and retrieval of chat history

Authentication & Authorization

The authentication layer uses cookie-based JWTs for both HTTP and WebSocket connections, ensuring seamless, secure access across all channels.

Login / Signup

Issues **accessToken** (stored as cookie) + refresh tokens for session continuity

WebSocket Handshake

Reads cookie and validates via **TokenService** on every connection

Authorization

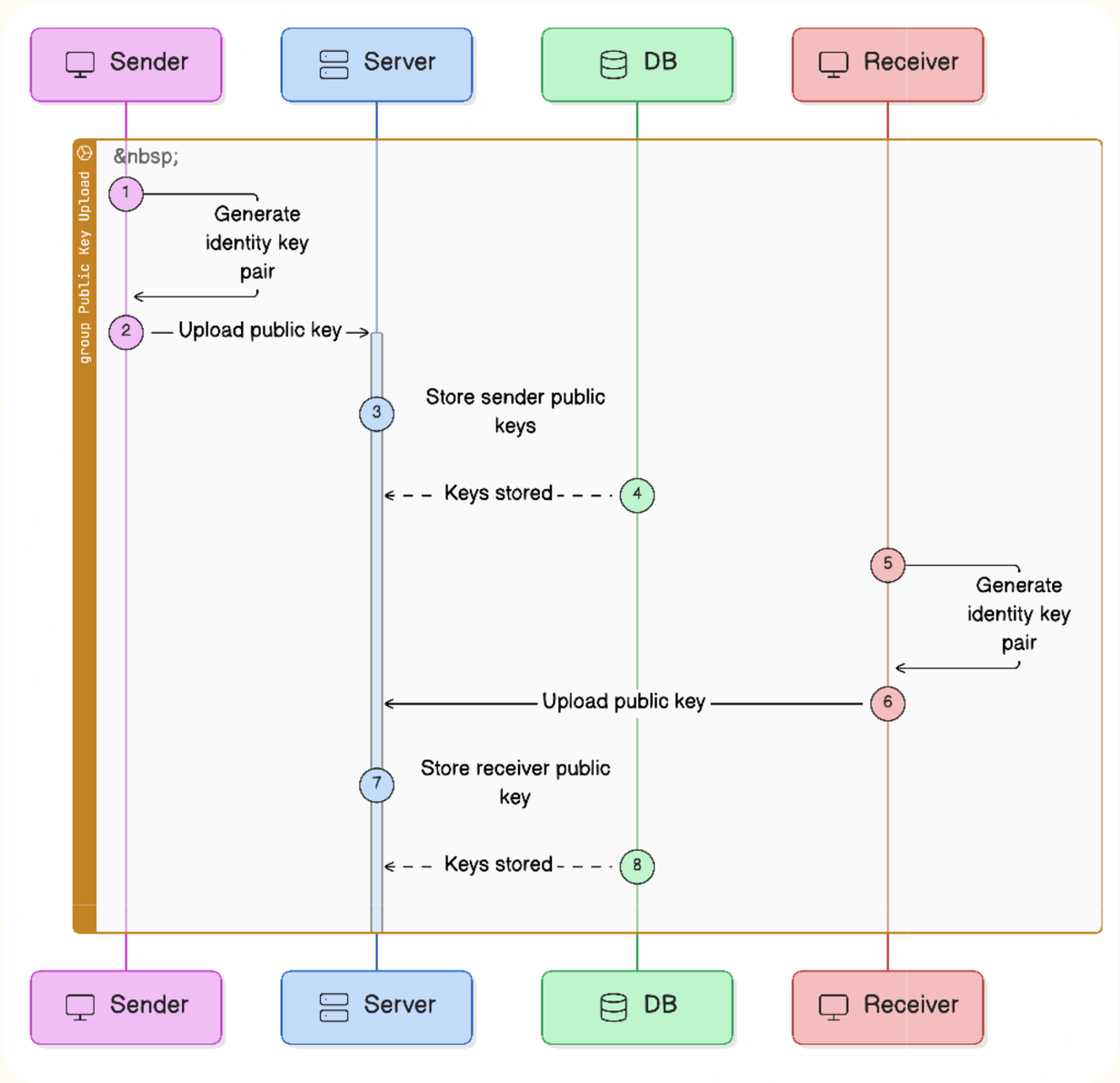
Checks membership via **conversationMember** table before granting room access

Recovery

Refresh tokens and **hashed recovery keys** supported server-side for secure account recovery

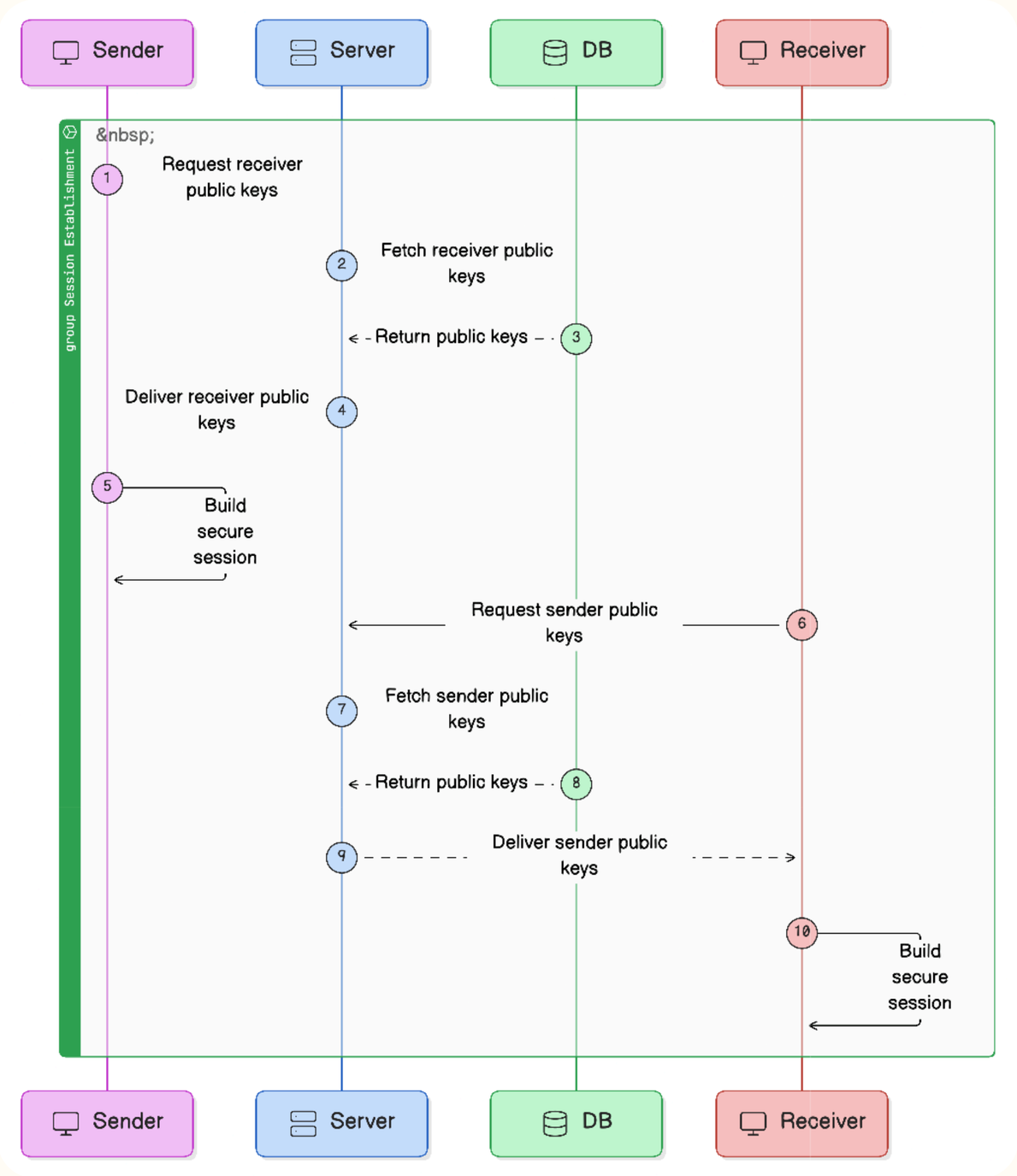
End-to-End Encryption Workflow

Each message is protected by a unique session key derived from long-term identity keys — the server never sees plaintext.



Public Key Upload

- Each user generates a unique identity key pair locally on their device.
- Only public keys are uploaded to the server; private keys never leave the client.
- The server securely stores public keys in the database for future session establishment.
- This process enables end-to-end encrypted communication without exposing private credentials.



Secure Session Establishment

- The sender requests the receiver’s public key from the server to initiate communication.
- The server retrieves and delivers the required public keys from the database.
- Both sender and receiver independently derive a secure encrypted session locally.
- The server acts only as a relay for public keys and cannot decrypt user messages.

Secure Real-Time Messaging Workflow

Key Exchange

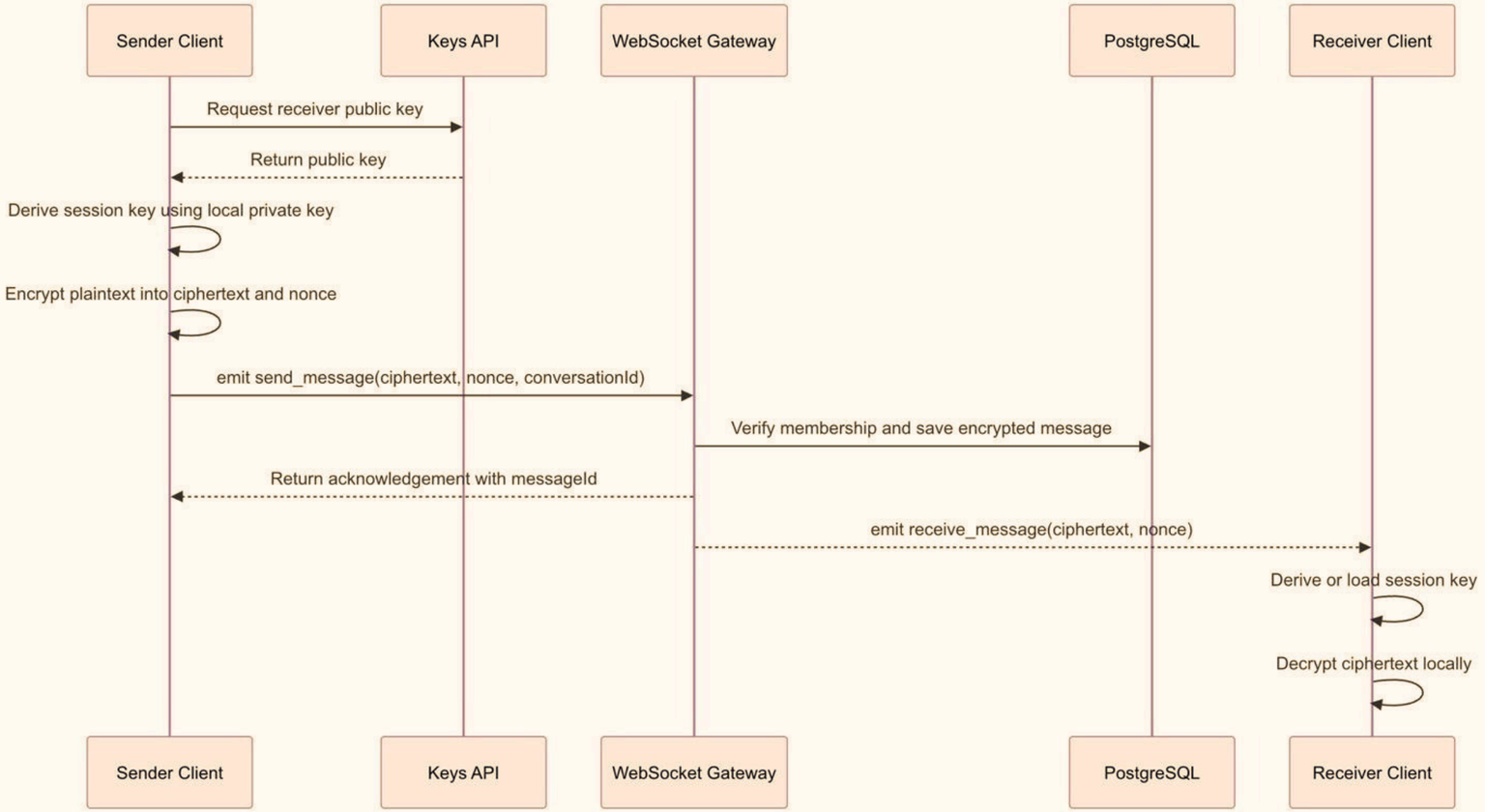
Client generates identity keypair; public key uploaded to server. Session keys derived per conversation via X3DH-like flow.

Socket Security

Socket.IO with cookie-based JWT auth. Rooms per conversation. TLS enforced in transit.

Message Encryption

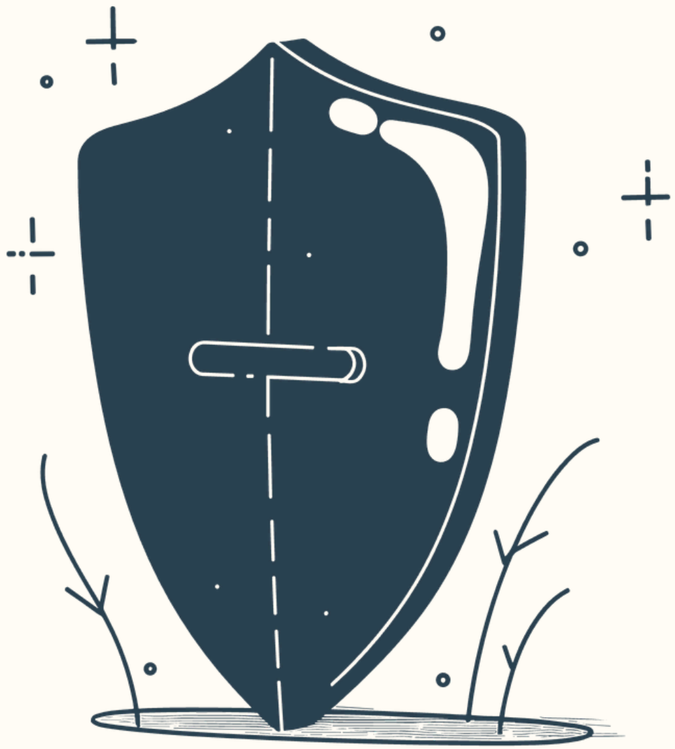
Uses XChaCha20-Poly1305 AEAD. Nonce stored alongside ciphertext in DB. Recovery key hashed for secure server-side verification.



Server Privacy Guarantees

Server stores only ciphertext + nonce. No plaintext messages. Recovery key hashed — never stored in plaintext.

Security Features



Client-Side Key Storage

- Identity keys stored securely in IndexedDB
- Private keys never transmitted to the server
- Users retain full control of cryptographic secrets

Access Control & API Security

- Route guards and WebSocket guards
- Conversation-level authorization checks
- CORS protection against unauthorized origins

Mnemonic-Based Recovery

- BIP-39 recovery phrases generated for account recovery
- Enables secure restoration of user identity
- Recovery credentials protected using bcrypt hashing

Authentication & Access Control

- JWT-based authentication with secure cookie storage
- Protected HTTP and WebSocket communication
- Conversation-level authorization and access validation

Future Enhancements

The roadmap prioritises features by security impact and implementation complexity, extending the messenger into a full-featured secure communication platform.

1

Group Conversations

Per-member ratcheting with advanced Signal-style double ratchet for multi-party E2EE

2

Encrypted Files

Encrypted file transfer and searchable encrypted metadata without exposing content to server

3

WebRTC A/V

Add audio/video calls with encrypted key exchange for end-to-end secured media streams

4

Mobile & Sync

Mobile clients with multi-device key sync and secure push notifications

Conclusion

This project demonstrates that strong privacy and a polished user experience are not mutually exclusive — even in a browser-based messenger.

E2EE Messenger

Fully functional web messenger with real-time delivery and XChaCha20-Poly1305 encryption

Ciphertext-Only Server

Server stores only ciphertext; all keys are client-owned and derived locally

Optimistic UX

Usable interface with optimistic UI and offline persistence via IndexedDB

Privacy Without
Footprints

Secure conversations with zero metadata collection and end-to-end encryption by default.

Create an Account

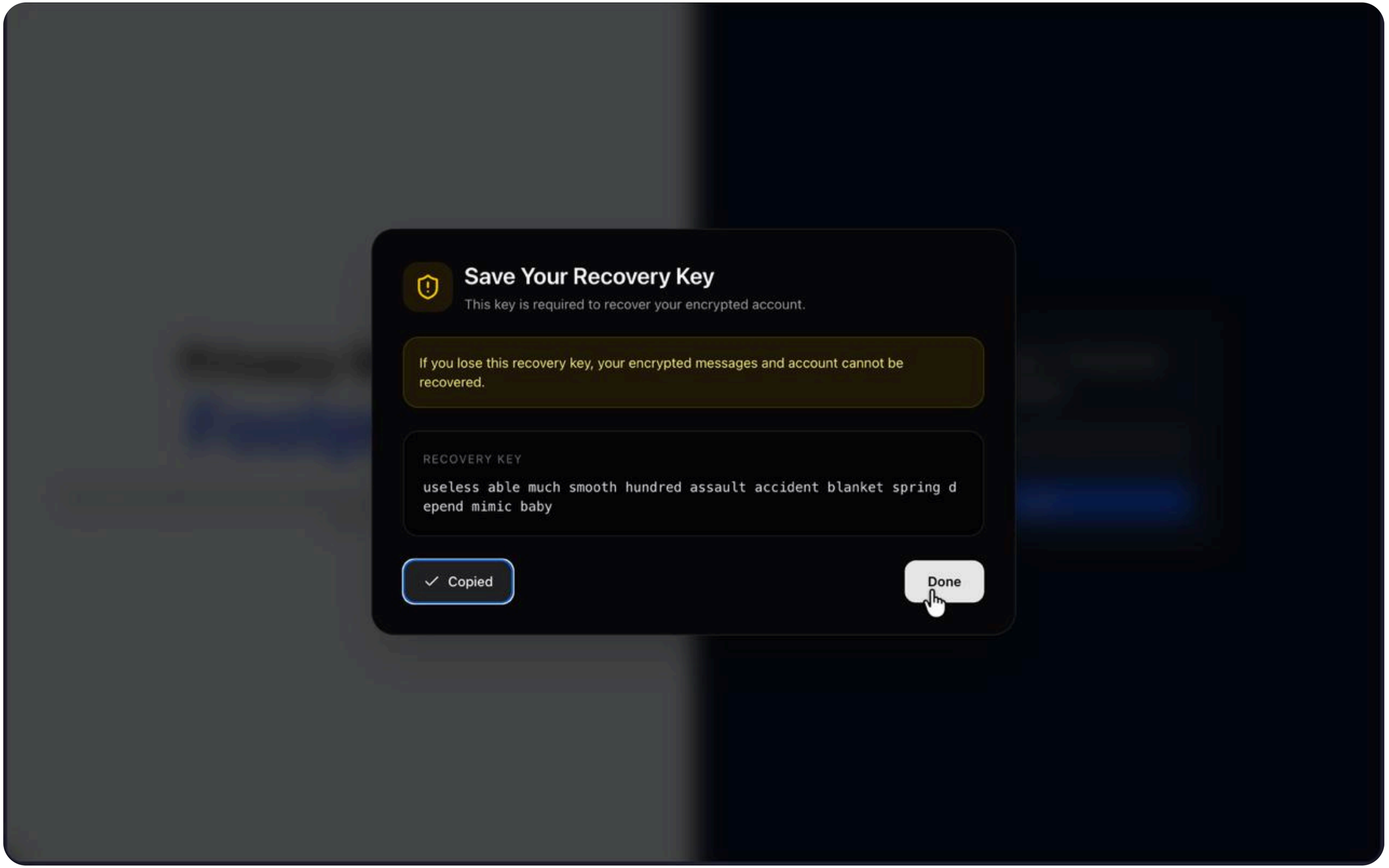
Login to existing account

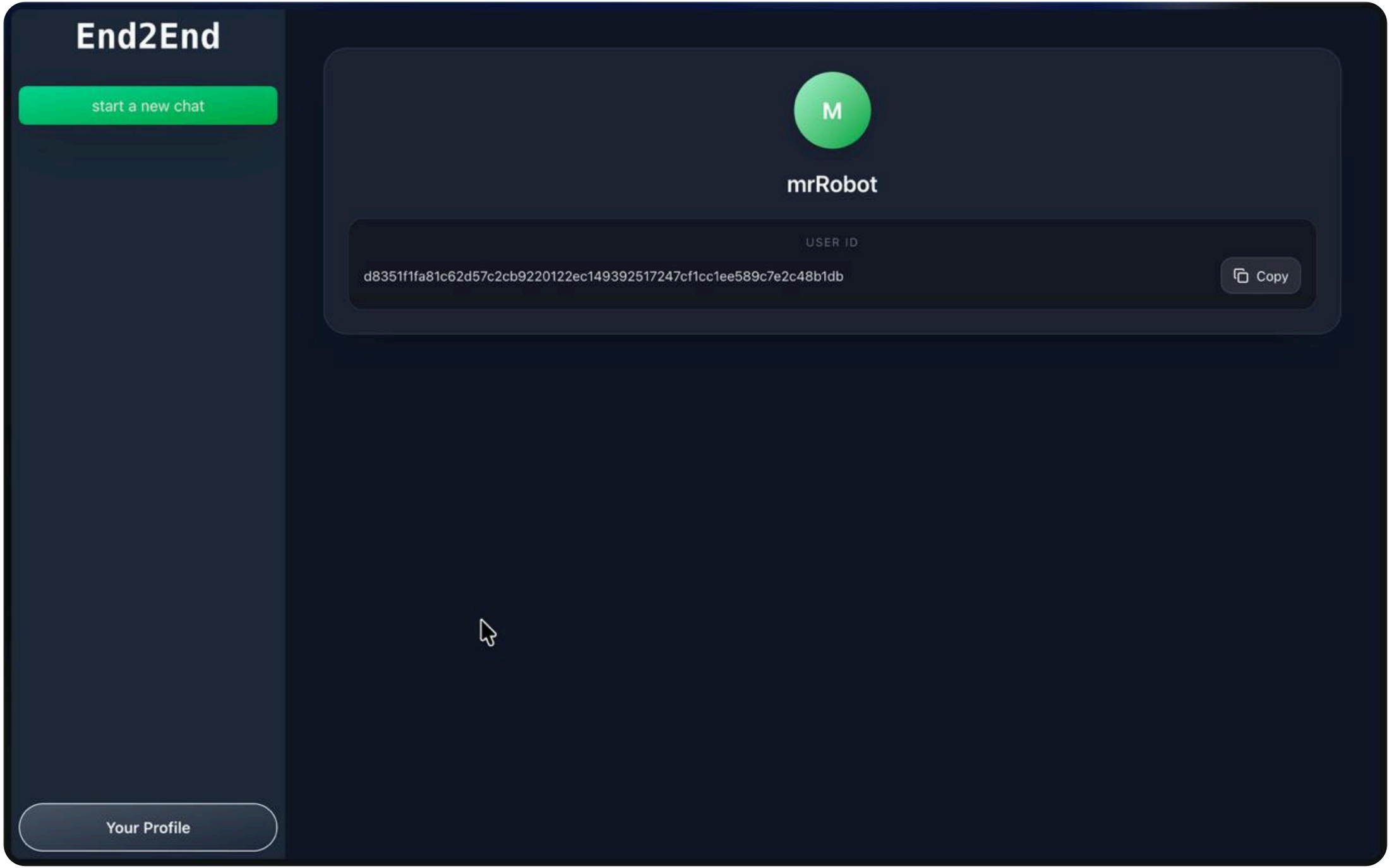
Privacy Without
Footprints

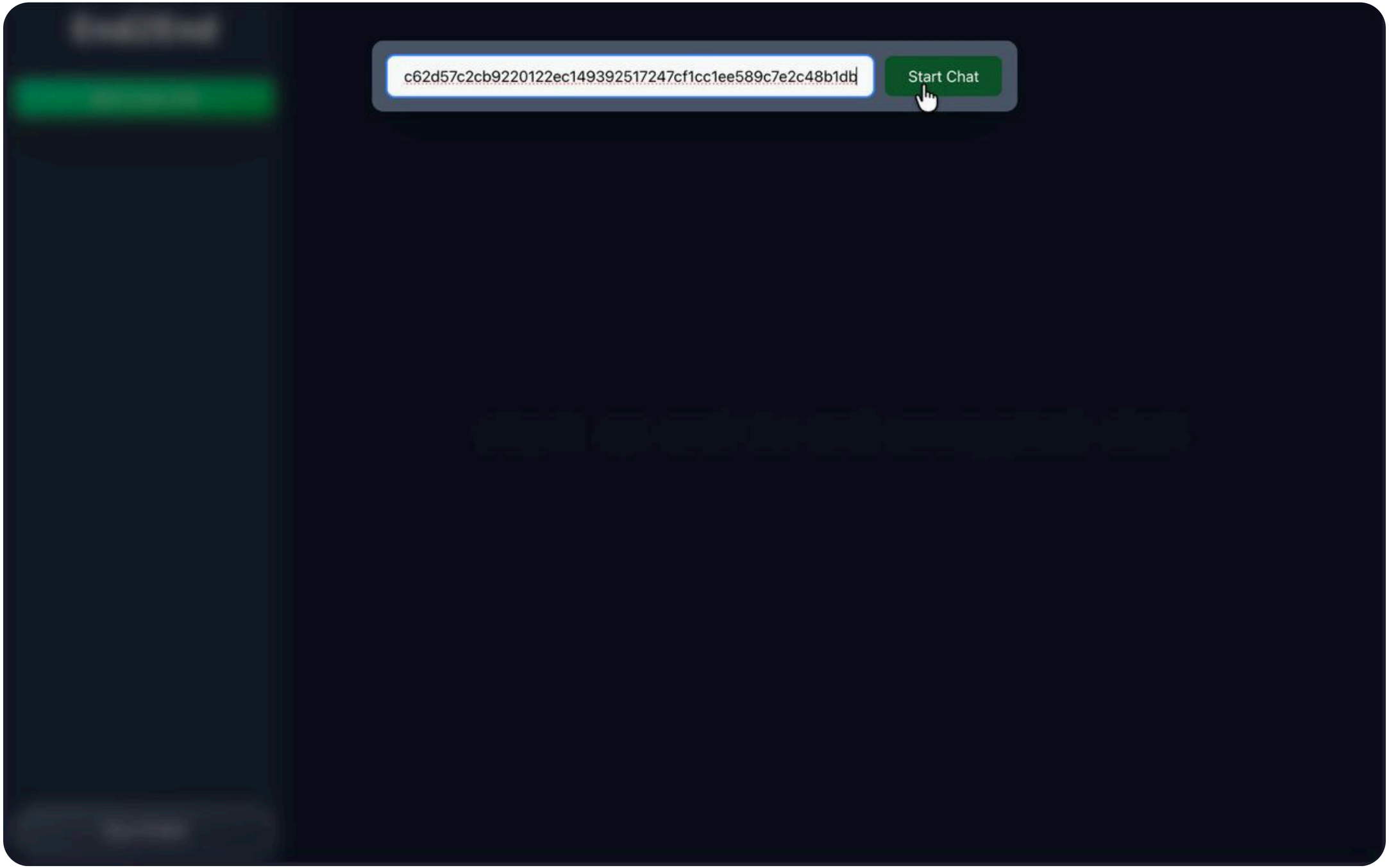
Secure conversations with zero metadata collection and end-to-end encryption by default.

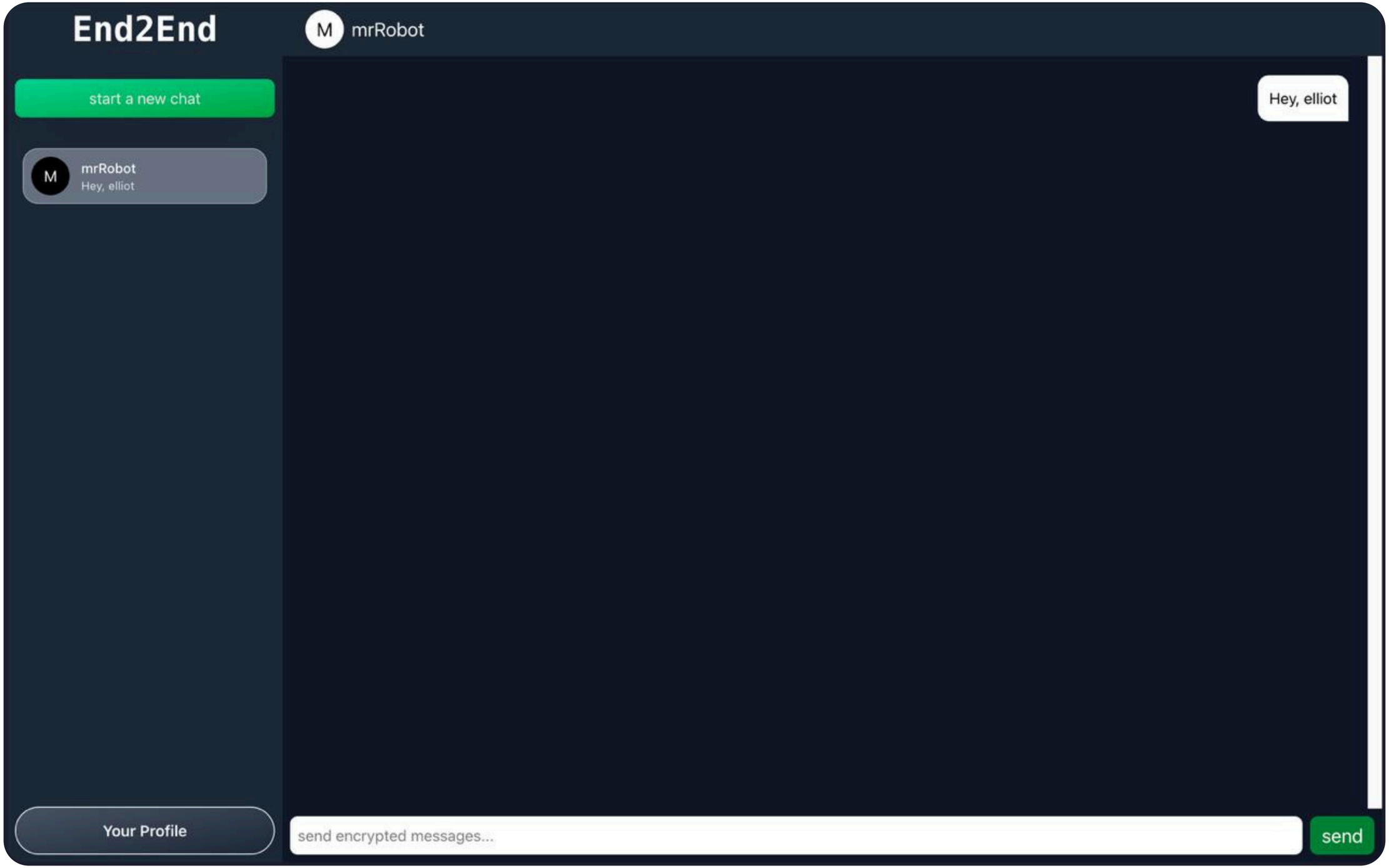
Privacy Without Footprints

Secure conversations with zero metadata collection and end-to-end encryption by default.


Save Your Recovery Key
This key is required to recover your encrypted account.
If you lose this recovery key, your encrypted messages and account cannot be recovered.
RECOVERY KEY
useless able much smooth hundred assault accident blanket spring depend mimic baby
Copied
Done


End2End
start a new chat
M
mrRobot
USER ID
d8351f1fa81c62d57c2cb9220122ec149392517247cf1cc1ee589c7e2c48b1db
Copy
Your Profile


c62d57c2cb9220122ec149392517247cf1cc1ee589c7e2c48b1db
Start Chat


End2End
M
mrRobot
start a new chat
M
mrRobot
Hey, elliot
Hey, elliot
Your Profile
send encrypted messages...
send

THANK YOU